

Project : Hotel Reservation System

Due Date: August 7th, 2026

Introduction

Hotel (Choose an appropriate name) is one of the famous tourist hotels in (Choose an appropriate name) city. But this hotel's current reservation is based on a manual system. When guests come in to make a reservation, their details are recorded in a file and then those files are stored in special cabinets. Also, the billing system is manual too.

As the current system is manual based, the management of the hotel must put extra efforts to keep the data secured. Records can easily be destroyed in case of fire, disaster or even possible to be stolen. Additionally, storing files requires extra cabinet space, and searching for a record is difficult, resulting in significant manpower hours. As well as the billing system, the system is manually handled so having an error in calculation is also at high risk. The management is also looking for a way to get the customer feedback after the stay.

Since the pandemic hits now the management is also looking to implement 2 kiosks as well, so the guests can book their room with no contact or interaction with anyone.

We have decided to develop their Reservation system as a computer-based system and hotel can give quick service to the guests.

Scope and outcomes

- **Goal:** Working in groups of 2 to 3, your team will build a desktop-only reservation and billing system that replaces manual processes and models real-world hotel operations with clear, maintainable architecture.
- **Architecture:** You will use MVC for presentation logic, a service layer for business rules, repositories powered by an ORM for persistence, and DI for wiring. You will apply Singleton, Strategy, Observer, Factory, and Decorator patterns where appropriate.
- **Deliverables:** Your team will submit combined project documentation, design artifacts, a working application, export files, logs, ORM configuration, database scripts, and a document containing both team and individual reflections on challenges and learnings.
- **Constraints:** You will not build any web components or use charts. All reports must be shown as tables and exportable to CSV, PDF, or TXT.

Architecture Tiers

This project must follow a **3-tier architecture**:

- **Presentation tier:**
 - JavaFX UI (kiosk, admin, feedback) with controllers and FXML views.
 - This tier collects input, validates at the UI level, and displays results in tables and forms.
- **Application/Business tier:**
 - Services implementing business rules, applying patterns, and orchestrating workflows.
 - This tier enforces occupancy rules, pricing, discounts, loyalty, and waitlist notifications.
- **Data tier:**
 - ORM-backed repositories and the relational database.
 - This tier manages persistence, queries, and transactions.

In addition, **cross-cutting concerns** such as logging, security, and configuration must be applied consistently across all tiers.

Functional requirements

Kiosk (self-service)

Welcome flow:

- The kiosk must display a brief, friendly welcome message and an optional short instructional video/ or gif.
- The rules and regulations button must always remain visible and accessible during the flow (like a navigation on the side).
- The interface must guide the user through a clear, step-by-step journey from arrival to confirmation.

Booking steps:

- The kiosk must ask for the number of adults and children before continuing.
- It must then ask for check-in and check-out dates and validate them immediately.
- Based on occupancy rules and room availability, the kiosk must either suggest a room plan and allow the user to adjust choices OR allow the user to choose their own type of rooms and quantity.

- Indicate the guest to check the rooms booking policy if user decide to choose their own type of room and quantity.
- The kiosk must collect guest details with visible required-field indicators and inline validation messages for each incorrect field.
- The kiosk must let the guest select add-on services such as Wi-Fi, breakfast, parking, and spa and must show the price impact for each selection.
- Before confirmation, the kiosk must present a complete estimate including subtotal, tax, add-ons and any loyalty effects.
- After confirmation, the kiosk must save the reservation and clearly inform the guest that billing will be handled at the front desk.

Validation:

- The kiosk must enforce occupancy limits per room type across all steps.
- It must accept a single-person booking without errors.
- It must reject any invalid combinations and must display clear, actionable error messages to the user.

Admin module

Authentication:

- The system must support multiple administrator accounts with role-based access (Admin and Manager).
- All passwords must be hashed with BCrypt before storage.
- The login process must provide success and failure feedback and must log all events.

Dashboard:

- Administrators must be able to search for guests and reservations by name, phone, date range, status, and other relevant filters.
- The dashboard must show results in paginated tables with sortable columns and must allow opening detailed views for editing.
- Reservations: Administrators must be able to create (via phone), modify, and cancel reservations while performing conflict checks against existing bookings.
- The system must support group bookings where a single reservation can include multiple rooms, and it must maintain a unified bill for the group.
- Payments: Administrators must be able to process payments using cash, card, or loyalty points.

- The system must support deposits at booking time, partial payments during the stay, and refunds when required.
- The system must track paid and outstanding balances and must prevent checkout while a balance remains.
- The system must offer loyalty program to the guest and if guest wants to enrolled for it then use the user information which is filled already confirm it with the guest and issue a loyalty number.

Discounts:

- Administrators must be able to apply discounts with role-based caps, where Admin can apply up to 15% and Manager can apply up to 30%.
- The system must prevent discounts that exceed configured limits and must record who applied each discount.

Checkout:

- Administrators must be able to generate the final bill, settle the balance, and mark the rooms as available.
- The system must trigger room availability notifications after checkout and must remind the administrator to invite the guest to submit feedback at the kiosk.

Waitlist:

- When rooms are unavailable, administrators must be able to add guests to a waitlist with their desired room type and date range.
- The system must notify subscribed administrators when availability changes and must provide a quick conversion from a waitlist entry to a reservation.

Feedback management:

- Administrators must be able to view feedback entries only after the guest has checked out.
- The system must provide filters by rating, date, sentiment tag, and guest.
- Administrators must be able to export feedback summaries for analysis.

Loyalty:

- Guests must earn loyalty points based on payment amounts, using a configurable earning rate.
- Administrators must be able to redeem points for discounts under defined caps. The system must provide a loyalty dashboard showing balances, earning history, and redemption activity.

Feedback (guest)

Submission:

- Guests must be able to submit a star rating from one to five along with comments after checkout.
- The system must store the feedback linked to both the reservation and the guest and must show a confirmation after submission.

Reporting (tabular only)

Revenue reports:

- The system must provide revenue summaries by day, week, and month.
- Each summary must include reservation counts, subtotal, tax, discounts, and total amounts.
- Reports must be displayed in tables and must be exportable to CSV or PDF.

Occupancy reports:

- The system must provide occupancy tables for daily, weekly, and monthly views.
- Each table must include rooms available, rooms occupied, and occupancy percentage as a numeric value only. Reports must be exportable to CSV or PDF.

Activity logs:

- The system must provide a table of administrative activity that includes timestamp, actor, action, entity type, entity identifier, and message.
- The system must support export to CSV and TXT.

Business rules

Occupancy limits:

- A single room must allow up to two people.
- A double room must allow up to four people.
- Deluxe and penthouse rooms must allow up to two people with higher base prices.
- The system must validate occupancy both per room and across group bookings.

Group booking suggestions:

- For groups of three or four adults, the system must suggest either one double room or two single rooms Or let them choose their own type of rooms and quantity.
- For groups larger than four adults, the system must suggest multiple double rooms or a combination of double and single rooms until capacity is satisfied.

- In the case of group booking choosing their own room types and quantity, system must validate the rules of occupancy.

Dynamic pricing:

- The system must apply a configurable multiplier for weekends and a separate multiplier for weekdays.
- The system must apply seasonal multipliers for defined date ranges such as peak season.
- The system must price add-ons either per night or per reservation, based on their pricing model.

Payments:

- The system must support cash, card, and loyalty point payments.
- The system must allow deposits at booking and must track partial payments during the stay.
- The system must allow refunds as negative payment entries and must adjust totals and logs accordingly.

Discounts:

- The system must enforce role-based discount caps and must prevent exceeding the configured limits.
- The system must cap loyalty points redemption per reservation and must apply discounts before loyalty redemption where required by the strategy.

Loyalty:

- The system must earn points per paid amount using a configurable rate.
- The system must redeem points into discounts using the loyalty strategy and must respect configured redemption caps.
- The system must maintain accurate point balances with audit trails for earning and redemption.

Feedback eligibility:

- The system must allow feedback submission only after the reservation has been checked out and any balances have been fully settled.

Architecture and patterns

Layers and packages:

- The project must organize code into app (bootstrap and DI), config (pricing and policy), controller (JavaFX), view (FXML and CSS), model (entities and enums),

service (business logic), repository (ORM-backed persistence), security (authentication and roles), util (logging and exporters), and events (observer components).

ORM and lifecycle:

- Entities must be annotated with JPA, including identifiers and relationships.
- The EntityManagerFactory must be created once and treated as a singleton or DI-managed singleton.
- The EntityManager must be created per transaction or unit of work and must not be shared across threads.

Repository abstraction:

- Repositories must expose clear interfaces and use JPA queries or criteria for persistence operations.
- Services must depend on repositories, not on ORM APIs directly.

Dependency injection:

- Controllers, services, and repositories must use constructor injection.
- A central configuration class must wire dependencies and provide singletons where appropriate.

Required patterns:

- Strategy must be used for billing calculations including standard, discount, and loyalty strategies.
- Observer must be used to notify administrators when room availability changes.
- Factory must be used to create room instances with configured attributes.
- Decorator must be used to add services such as spa, breakfast, Wi-Fi, and parking to booking pricing.

Logging, security, and validation

Activity logging:

- The application must record administrator actions including logins, searches, reservation changes, checkouts, cancellations, discounts, payments, refunds, and feedback submissions.
- Each log entry must include a timestamp, actor, action, entity type, entity identifier, and a descriptive message.

Logger configuration:

- The application must use `java.util.logging` or Log4j with a rotating file handler.

- Each log file must be limited to approximately one megabyte, and the system must retain up to ten files before rotating.
- Students should configure logging to store logs in a **separate log file**.
- Consider implementing **log rotation** to avoid excessive file size growth.

Example configuration and rotation (using FileHandler):

```
try {
    FileHandler fileHandler = new FileHandler("system_logs.%g.log", 1024 * 1024, 10, true);
    logger.addHandler(fileHandler);
    SimpleFormatter formatter = new SimpleFormatter();
    fileHandler.setFormatter(formatter);
} catch (IOException e) {
    logger.log(Level.SEVERE, "Failed to initialize logger", e);
}
```

- "system_logs.%g.log" is a pattern where %g is a placeholder for the generation number of the log file.
- 1024 * 1024 sets the limit to 1MB for each log file.
- 10 specifies the maximum number of log files to keep.
- This setup will create a new log file after the current file reaches 1MB and will maintain up to 10 log files. Anything beyond that will overwrite the oldest log file.

Exception logging:

- The application must log validation failures, persistence errors, and unexpected exceptions at appropriate levels.
- Severe issues must include stack traces for troubleshooting.

Authentication and authorization:

- The application must store only BCrypt-hashed passwords and must perform role checks for sensitive actions such as discounts, refunds, reporting, and user management.

Validation rules:

- The application must validate guest names, phone numbers, and email addresses with clear messages.

- The application must validate date ranges with minimums and must check for overlaps.
- The application must validate occupancy distribution across group bookings.
- The application must validate payment amounts and must prevent negative balances.
- The application must validate discounts within configured caps and must enforce non-negative values.
- The application must validate feedback ratings and must cap comment length.

Reporting specifications

Revenue reports:

- The system must show period, reservation count, subtotal, tax, discounts, and total for day, week, and month views.
- The system must support filtering by date range and room type and must export to CSV or PDF.

Occupancy reports:

- The system must show date, rooms available, rooms occupied, and occupancy percentage in numeric form.
- The system must support filtering by date range and room type and must export to CSV and PDF.

Feedback summary:

- The system must show reservation identifier, guest, rating, comment, date, and sentiment tag.
- The system must display the average rating and counts for common issue tags.
- The system must export to CSV.

Activity logs:

- The system must show timestamp, actor, action, entity type, entity identifier, and message.
- The system must read from the log file or the audit table and must export to CSV or TXT.

- **UI Prototype:** High-fidelity screenshots of the Kiosk (Welcome, Room Selection, Add-ons) and Admin Dashboard complete all screens.
- **Domain Modeling:** A Class Diagram identifying core entities (Guest, Room, Reservation, Payment) and their relationships (One-to-Many, etc.).
- **Behavioral Modeling:** A **Sequence Diagram** showing the "Booking Flow" to demonstrate how the UI Controller interacts with the Service and Repository layers.
- **Database Schema:** An ERD showing primary/foreign keys and constraints.
- Separate submission will be open where you need to submit all UML, front end screen shots, database designs, or any other design diagrams that you have shown in the lab. (Only those submissions will be accepted which are approved during the lab time).
- Attendance is mandatory for this milestone.

Note: Students can show their design in earlier labs as well.

- **The Persistence Sprint:** Implementation of the **ORM/JPA** layer, including the Singleton EntityManagerFactory.
- **The Kiosk Path:** A guest must be able to complete a booking from start to finish, with data successfully persisting to the database.
- **Factory Pattern:** To generate different room types (Single, Double, Penthouse).
- **Strategy Pattern:** For basic price calculations (Standard vs. Weekend).
- **Navigation:** All Admin buttons must work to "open" windows, even if the internal logic (like processing refunds) isn't finished.
- Your team must upload a single combined video (minimum 3 minutes) discussing your full database plan (i.e., all the tables, relationships, etc.). Each team member must speak during the video and clearly explain the specific database entities, JPA annotations, or relationships they were responsible for designing.
- Uploads
 - Database design diagrams.
 - Zipped project
 - Video

Note: not all the buttons needed to be fully functional, but they should open the targeted window and then switch back to main window upon closing.

- Admin Functionality: Full CRUD for reservations, role-based discount caps (15% vs. 30%), and the Loyalty system.
- Observer: Admins receive notifications when rooms become available.
- Decorator: Dynamically adding services (Spa, Wi-Fi) to the bill
- Cross-Cutting Concerns: * Security: Password hashing with BCrypt.
- Logging: Rotating file logs (1MB limit, 10-file rotation).
- Reporting: Tabular data exports to CSV or PDF for revenue and occupancy.
- The last deadline is your final team submission. Your team must record a combined video demonstrating the fully working project. Each team member must actively participate in the demonstration, specifically showcasing the architectural tiers (Presentation, Business, or Data) or features they developed, and briefly explaining a technical challenge they overcame.
- The Reflection document must include:
 - Team Challenges: A summary of overall project hurdles (e.g., merging code, integrating the front-end with the ORM) and how the team resolved them.
 - Individual Reflections: Clearly labeled sections for each group member where they describe their specific coding challenges, solutions, and personal learnings during the project.

Project Documentation

Purpose of Your Documentation

This is not just a summary of the project instructions — it's your opportunity to explain how you understood, designed, and implemented the system. Your documentation should reflect your decisions, challenges, and learning journey. It should include your own diagrams, explanations of how you applied patterns and business rules, and reflections on what worked well and what you'd improve. Think of it as a professional walkthrough of your work, showing how you brought the project to life.

Documentation Checklist

1. Project Overview

- ☐ Summary of system and purpose
- ☐ Key features
- ☐ Technologies used

2. Architecture Summary

- ☐ Description of 3-tier architecture
- ☐ Cross-cutting concerns
- ☐ MVC, DI, ORM usage

3. Design Artifacts

- ☐ Class diagram
- ☐ Sequence diagrams
- ☐ Optional UI screenshots

4. Entity and Relationship Mapping

- ☐ List of entities
- ☐ Relationships and annotations
- ☐ Cascade/fetch/validation notes

5. Pattern Usage

- ☐ Strategy, Observer, Factory, Decorator, Singleton
- ☐ Where and how each is used

6. Business Rules

- ☐ Occupancy, pricing, discounts, loyalty, feedback
- ☐ Enforcement logic

7. Security and Logging

- ☐ Authentication and roles

- ☐ Logging configuration and samples
- ☐ Exception handling

8. Export and Reporting

- ☐ Report types and formats
- ☐ Optional sample exports

9. Challenges and Learnings

- ☐ Overall team technical challenges
- ☐ Individual reflections from each team member
- ☐ Suggestions for improvement

Submit one PDF or DOCX file named ProjectDocumentation_.pdf



Grading Rubric

- **Design & Architecture (20%)**
MVC separation, DI, package structure, UML diagrams
- **Patterns & Principles (15%)**
Strategy, Observer, Factory, Decorator, Singleton; OO principles
- **ORM & Persistence (15%)**
JPA annotations, relationships, EMF/EM usage, queries
- **Functionality – Kiosk (10%)**
Booking flow, validation, dynamic pricing
- **Functionality – Admin (15%)**
Login, search, modify, payments, checkout, notifications
- **Functionality – Waitlist & Loyalty (10%)**
Waitlist creation, observer notifications, loyalty dashboard
- **Reporting & Feedback (10%)**
Tabular reports, export formats, feedback flow
- **Logging & Security (5%)**
Logger rotation, audit logs, exception handling, BCrypt

● **Passing threshold:** Minimum 50% overall, with working kiosk booking, admin login, and ORM persistence

Note: Every Wednesday lab students can approach and discuss the design of the project, their progress and if there are any issues.

Optional Requirement: Implementing a Multithreaded Server for Admin Access

In a real-world hotel reservation system, multiple admins might need to log in and manage bookings at the same time. To simulate this, you must implement a multithreaded server that allows multiple admin sessions to run simultaneously.

What This Means:

- The system should allow at least two admins to log in and use the system at the same time by running multiple instances of your JavaFX Admin application.
- Each admin should be able to interact with their respective JavaFX UI to search guests, modify reservations, process checkouts, and apply discounts independently.
- The server should handle multiple admin requests without conflicts or crashes.

How to Achieve This?

1. Use a Multi-Threaded Server Approach

- The system should have a server-side application that listens for admin connections.
- When an admin logs in, the server should create a new thread to handle that admin's session.

Example:

```
class AdminHandler extends Thread {  
    private Socket socket;  
    public AdminHandler(Socket socket) {  
        this.socket = socket;  
    }  
    public void run() {  
        // Handle admin requests here  
    }  
}
```

- This ensures that multiple admins can work independently without affecting each other.

2. Client-Server Communication

- Your JavaFX Admin UI (Presentation Tier) must act as the client. Instead of the UI controllers calling the Service/Database tiers directly in local memory, the controllers must send requests (e.g., serialized objects or JSON) over a Socket to your central server.
- The backend server will process these requests, interact with the ORM/Database, and send responses back to the specific admin's JavaFX instance.

3. Testing the Multithreaded Functionality

- Start your backend Java Server application first.
- Launch two separate instances of your JavaFX Client application on your machine.
- Verify that two distinct admins can log in to the different UI windows, search for guests, and process reservations simultaneously without causing data conflicts or server crashes.

Good Luck and Happy Coding....